

NexusLoss

Teknik referans · sürüm 0.1.0

Bu proje, Bursa Teknik Üniversitesi Bilgisayar Mühendisliği 1. sınıf öğrencisi **Muhammed Fatih Şahin** tarafından geliştirilmiştir.

NexusLoss, eğitim adımıyla tahmin ile hedef arasındaki hatayı ve bu hatanın tahmine göre türevini üretir. C++20, header-only, çalışma anında bağımlılıksız. Hesap CPU üzerinde, *float* ve *double* şablonlarıyla, analitik gradyanla yürür. On üç aile dokümantasyon sitesinde 88 kayıt olarak indekslenir.

C++20

header-only

std::span

analitik gradyan

CPU

13 aile

İçindekiler

1. Projenin rolü
2. İçinde ne var
3. Performans mimarisi
4. Klasör ve dosya yapısı
5. Kurulum ve hızlı başlangıç
6. Sözleşme
7. İndirgeme ve ağırlık
8. Sayısal kararlılık
9. Hangi kaybı seçmeli
10. Kayıp kataloğu
11. Şekil ve kodlama
12. Test ve dokümantasyon

Bu basım, depo kökündeki `README.md` ile aynı sözleşmeyi taşır. Kaynak sayfa `docs/NexusLoss.html` dir. Gezinmeli anlatım `docs/website` içindeki sitededir.

1. Projenin rolü

Bir eğitim adımı dört parçaya ayrılır. Veri gelir, model bir tahmin üretir, kayıp o tahmini hedefle karşılaştırır, eniyileyici ağırlığı gradyan yönünde günceller. NexusLoss yalnızca üçüncü parçadır.

Çağırır verir

- tahmin, `std::span<const T>`
- hedef, aynı uzunlukta span veya bir sınıf indeksi
- isteğe bağlı örnek ağırlığı

Kütüphane döndürür

- skaler ya da eleman bazlı kayıp
- tahminle aynı uzunlukta `dL / d(prediction)`

Dönen vektör bir autograd düğümü değildir. Çağırır bu vektörü modele geri yayar. Kütüphane ağırlık tensörünü görmez, öğrenme hızı uygulamaz, veri yüklemes.

İki hesap şekli

Eleman bazlı kayıplar `LossBase<T>` üzerinden gelir. Regresyon, ikili çapraz entropi ve hinge ailesi bu gruptadır. `compute_element_wise` her konum için bir hata üretir, `core::apply_reduction` bunu özetler, `gradient` aynı özete uygun ölçeği uygular.

Yapısal kayıplar `LossBase` tan türemez. Kutu, CTC, nokta bulutu, liste sıralaması ve tek örnekli çok sınıflı çapraz entropi kendi `forward / backward` imzasını kullanır. Şekil bilgisi (zaman, sınıf, kutu köşesi, nokta boyutu) düz vektörde kaybolmaz.

Nexus içindeki yeri

Parça	İşi
NexusData	veri
NexusModel	katman ve tahmin
NexusLoss	kayıp değeri ve tahmine göre gradyan
NexusOptim	ağırlık güncellemesi
NexusTrain	adımın sırası: forward, sonra backward
MatrixFlash Pro	ayrı autograd grafiği. NexusLoss bu grafa bağlanmaz

Üst katman, dönen skaler ve gradyanı isterse `matrix_pro::custom_loss` içine koyar. Derleme bağımlılığı yoktur.

2. İçinde ne var

On üç aile, başlık dosyalarında sınıf ve serbest fonksiyon olarak durur. Dokümantasyon sitesi bu kümeyi **88 kayıt** olarak indeksler. Kayıtların bir kısmı sınıf, bir kısmı aynı formülün serbest fonksiyonudur. Asıl kaynak `include/nexusloss/losses/` altındaki başlıklardır.

Aile	Ad alanı	Ne ölçer
Regresyon	<code>nexusloss</code>	sürekli tahmin ile hedef
Sınıflandırma	<code>nexusloss</code> ve <code>classification</code>	olasılık veya logit ile etiket
Segmentasyon	<code>segmentation</code>	maske örtüşmesi ve sınır
Tespit	<code>detection</code>	kutu örtüşmesi, odak, kutu regresyonu
Metrik öğrenme	<code>metric</code>	gömme yakınlığı ve açısal marj
Sıralama	<code>ranking</code>	çift, liste, yaklaşık NDCG
Üretken	<code>generative</code>	çekişmeli kayıp, ELBO, difüzyon
Öz-denetimli	<code>self_supervised</code>	görüntü çiftleri, çapraz korelasyon
Damıtım	<code>distillation</code>	öğretmen logit ve öznetelik
Dizi	<code>sequence</code>	CTC, maskeli token, sonraki cümle
Nokta bulutu	<code>point_cloud</code>	Chamfer ve eşit kardinaliteli EMD
Pekiştirme	<code>reinforcement</code>	politika, PPO, değer, entropi
Sağkalım	<code>survival</code>	Cox kısmi olabilirlik, Weibull

Çekirdek iki dosyadır. `core/reduction.hpp` içinde `Mean`, `Sum`, `None`, `BatchMean` vardır. `core/utils.hpp` boyut kontrolü, `epsilon` (`1e-7`), `safe_log`, taşmayan sigmoid, `log_sum_exp`, `softmax`, `one_hot`, maske, `safe_sqrt` ve `safe_divide` tutar.

Tek giriş `nexusloss/nexusloss.hpp` dir. Bu başlık hesap yapmaz; çekirdeği ve on üç aileyi birlikte açar.

3. Performans mimarisi

Hız, iş parçacığı havuzundan veya elle yazılmış SIMD çekirdeğinden gelmez. Döngüler skaler ve taşınabilir. - 03 altında derleyici bunları otomatik vektörleyebilir; kütüphane buna söz vermez. Kazanç, bağımlılıksız ve tahminsiz bir sıcak yoldur.

Karar	Etkisi
Header-only şablon	<code>float</code> ve <code>double</code> ayrı derlenir. Sanal çağrı eleman başına değildir; <code>LossBase</code> sanal fonksiyonu vektörün tamamı için bir kez çalışır, aritmetik düz döngüdür.
<code>std::span</code>	<code>compute</code> ve <code>gradient</code> çağırının tamponunu kopyalamadan okur. Sahiplik çağıranda kalır.
Analitik gradyan	Eğitim yolunda sonlu fark yoktur. Sonlu fark yalnızca testtedir: adım $1e-6$, tolerans $1e-5$.
Autograd şeridi yok	Ara tensör ve topoloji tutulmaz. Bellek, giriş ve çıkan gradyan kadardır. Kütüphane gradyanı katmanlara dağıtmaz.
<code>forward</code> önbellegi	Tahmin, hedef ve ağırlık kopyalanır ki <code>backward()</code> argümansız çağırılsın. Kopya istemeyen yol <code>gradient(pred, target)</code> dir.
İki API şekli	Kutu, CTC ve nokta bulutu eleman bazlı tabana zorlanmaz. Skaler artı şekilli gradyan döner.
Taşma kontrolü	<code>log_sum_exp</code> maksimumu çıkarır. Sigmoid büyük logitlerde ayrı dal kullanır. Log ve bölme <code>epsilon</code> tabanına oturur.
İndirgeme ölçeği	<code>Mean</code> gradyanı $1/N$ ile böler. <code>Sum</code> bölmez; aynı öğrenme hızıyla adım yaklaşık N kat büyür.
Test bağımlılığı	<code>GoogleTest</code> yalnızca testler açıkken indirilir. Uygulama <code>gtest</code> 'e bağlanmaz.

Derleyici uyarıları arayüzle birlikte gelir: MSVC'de `/W4 /utf-8`, diğerlerinde `-Wall -Wextra -Wpedantic`. Standart uzantıları kapalıdır.

4. Klasör ve dosya yapısı

```
NexusLoss/
├── CMakeLists.txt           sürüm 0.1.0, INTERFACE hedef
├── README.md
├── include/nexusloss/
│   ├── nexusloss.hpp       tek include
│   ├── core/reduction.hpp  Mean, Sum, None, BatchMean
│   ├── core/utils.hpp      kararlılık yardımcıları
│   └── losses/
│       ├── loss_base.hpp   eleman bazlı sözleşme
│       ├── regression.hpp
│       ├── classification.hpp
│       ├── segmentation.hpp
│       ├── detection.hpp
│       ├── metric.hpp
│       ├── ranking.hpp
│       ├── generative.hpp
│       ├── self_supervised.hpp
│       ├── distillation.hpp
│       ├── sequence.hpp
│       ├── point_cloud.hpp
│       ├── reinforcement.hpp
│       └── survival.hpp
├── examples/main.cpp       Huber, BCE-with-logits, çapraz entropi
├── tests/                  aile başına bir cpp + grad_check.hpp
└── docs/
    ├── NexusLoss.pdf       bu referans
    ├── NexusLoss.html      basım kaynağı
    └── website/             Next.js dokümantasyon sitesi
```

`build/` ve `cmake-build-debug/` CMake üretimidir. `docs/website` ayrı bir Node projesidir; C++ derlemesine girmez. CMake kurulum hedefi (`install`) tanımlı değildir. Tüketici `add_subdirectory` kullanır ya da `include/` yolunu derleyiciye verir.

5. Kurulum ve hızlı başlangıç

İsterler: CMake 3.20+, C++20 derleyicisi. Kütüphanenin kendisi için ağ gerekmez. Testler açıkken GoogleTest v1.14.0 yapılandırma sırasında indirilir. `NEXUSLOSS_BUILD_TESTS` ve `NEXUSLOSS_BUILD_EXAMPLES` öntanımlı açıktır.

```
add_subdirectory(path/to/NexusLoss)
target_link_libraries(uygulama PRIVATE nexusloss::nexusloss)
target_compile_features(uygulama PRIVATE cxx_std_20)
```

```
cmake -S . -B build -DNEXUSLOSS_BUILD_TESTS=ON -DNEXUSLOSS_BUILD_EXAMPLES=ON
cmake --build build --config Release
ctest --test-dir build -C Release --output-on-failure
```

Tek yapılandırılmalı üreteçlerde `Release` alt klasörü olmaz. Örnek ikili `basic_example` dir.

Üç çağrı

```
#include <nexusloss/nexusloss.hpp>
#include <vector>

const std::vector<double> prediction{2.0, 4.0, 3.0};
const std::vector<double> target{1.0, 5.0, 2.5};
nexusloss::HuberLoss<double> huber(1.0);
const double value = huber.forward(prediction, target).front();
const std::vector<double> gradient = huber.backward();

nexusloss::BCEWithLogitsLoss<double> bce;
const double bce_loss = bce.forward(logits, binary_target).front();

nexusloss::classification::CrossEntropyLoss<double> ce;
const double ce_loss = ce.forward(class_logits, /*sınıf*/ 2);
const std::vector<double> ce_grad = ce.backward();
```

`forward(...).front()` yalnızca `Mean` ve `Sum` için kaybın tamamıdır. `None` vektörü girişle aynı uzunluktadır. Çok sınıflı `CrossEntropyLoss::forward` doğrudan skaler döndürür.

```
nexusloss::MSELoss<float> mse;
const auto value = mse.compute(pred, y); // önbellek yok
const auto grad = mse.gradient(pred, y); // forward şart değil
```

6. Sözleşme

Çağrı	Girdi	Çıktı	Önbellek
compute	pred, target, isteğe bağlı weight	indirgenmiş kayıp	yazmaz
gradient	pred, target, isteğe bağlı weight	$dL/d(pred)$	yazmaz
forward	aynı	compute ile aynı	girdileri kopyalar
backward	yok	son forward'un gradyanı	okur

`compute_element_wise` ve `gradient_element_wise` indirgeme uygulamaz. Ölçek `gradient` içindedir. MSE eleman türevi $2(pred - target)$ dir; `Mean` bunu ayrıca $1/N$ ile çarpar.

- Boş span ve boyu tutmayan tensör `std::invalid_argument` atar.
- `backward()`, `forward` olmadan `std::logic_error` atar.
- Türevi yazılmamış kayıp `gradient_element_wise` içinde hata verir. `ZeroOneLoss` sıfır altgradyan döndürür.
- `BatchMean` düz vektörde `Mean` ile aynıdır. Satır bazlı kayıpta batch bölümü çağırandır.
- Çok sınıflı API tek örnektir. Batch döngüsü ve indirgeme çağırandır. Logit ham skordur; dışarıda softmax uygulayıp tekrar vermeyin.
- Serbest fonksiyonlar (`ciou_loss`, `info_nce_loss`, `cox_ph_loss`) nesne tutmaz. Gradyan için eşlenik `*_gradient` veya sınıfın `backward` u kullanılır.

7. İndirgeme ve ağırlık

Tip	Kayıp	Gradyan
Mean	toplam / N. Ağırlıklıysa payda $\sum(w)$, taban ϵ	eleman türevi $\times 1/N$ veya $\times w_i / \sum(w)$
Sum	toplam, ya da $\sum(w_i L_i)$	ölçek yok; ağırlık varsa w_i çarpanı
None	aynı uzunlukta vektör	her eleman kendi kaybının türevi
BatchMean	düz vektörde Mean	Mean ile aynı ölçek

Sınıf dengesinde `classification::effective_number_class_weights` etkin sayı formülüyle sınıf ağırlığı üretir. `weighted_categorical_cross_entropy` ve `weighted_binary_cross_entropy` bu ağırlığı kayba taşır.

8. Sayısal kararlılık

`core::epsilon<T>` değeri `1e-7` dir. `float` ve `double` aynı eşiği kullanır.

Yardımcı	Görevi
<code>safe_log</code>	$\log(\emptyset)$ ve negatif log
<code>clamp_probability</code>	olasılığı (<code>eps</code> , <code>1-eps</code>) içine alır
<code>sigmoid</code>	logit $\rightarrow$ olasılık, taşmayan <code>exp</code>
<code>log_sum_exp</code>	$m + \log(\sum \exp(x - m))$, $m = \max(x)$
<code>softmax</code>	dağılım, toplam 1
<code>safe_sqrt</code> / <code>safe_divide</code>	negatif kök ve sıfır payda
<code>one_hot</code>	sınıf indeksi $\rightarrow$ örnek-major düz satır
<code>apply_mask</code>	pad konumunu sıfırla çarpar

Poisson kaybı tahmin olarak log-oran bekler. Tweedie kaybı $1 < \text{power} < 2$ ve pozitif ortalama ile birim dağılımlı sapma hesaplar.

9. Hangi kaybı seçmeli

Problem	Başlangıç	Sapınca
Gerçek değerli regresyon	MSELoss	aykırı varsa HuberLoss (delta 1) veya MAELoss
Pozitif, çarpık hedef	MSLELoss	sayım ise PoissonLoss
Asimetrik maliyet	QuantileLoss	
İkili etiket	BCELoss	logit varsa BCEWithLogitsLoss
Tek doğru sınıf	CrossEntropyLoss	gürültülü etiket: label smoothing
Dengesiz sınıf	FocalLoss	effective_number_class_weights
Etiket ± 1	HingeLoss	BCE bu kodlamayı kabul etmez
Piksel maskesi	DiceLoss	TverskyLoss, FocalTverskyLoss
Kutu köşeleri	CIoULoss	yalnız örtüşme: IoULoss
Normalize kosinüs	ArcFaceLoss	üçlü örnek: TripletLoss
Sıralı liste	ListNetLoss	çift: RankNetLoss
Çekişmeli ağ	WassersteinLoss	alt sınır: VAEELB0Loss
İki görünüm	InfoNCELoss	BarlowTwinsLoss
Öğretmen logit	KnowledgeDistillationLoss	öznitelik: FeatureDistillationLoss
Hizalanmamış dizi	CTCLoss	maskeli token: MLM kaybı
İki nokta bulutu	ChamferDistanceLoss	küçük eşit bulut: EMD
Politika gradyanı	PolicyGradientLoss	PPOClipLoss, epsilon 0.2
Sansürlü süre	CoxPHLoss	WeibullNLLLoss

10. Kayıp kataloğu

Regresyon

Hepsi `LossBase<T>` tir. `PoissonNLLLoss<T>`, `PoissonLoss<T>` takma adıdır.

Sınıf	Özet
<code>MSELoss</code>	$(pred - target)^2$, gradyan $2(pred - target)$
<code>MAELoss</code>	mutlak fark, L1
<code>HuberLoss</code>	küçük hatada kare, büyük hatada doğrusal; delta öntanımlı 1
<code>LogCoshLoss</code>	$\log(\cosh(pred - target))$
<code>MSLELoss</code>	log uzayında kare
<code>QuantileLoss</code>	pinball
<code>PoissonLoss</code>	negatif log olabilirlik; tahmin log-oran
<code>TweedieLoss</code>	birim dağılımlı sapma, güç (1, 2)
<code>CauchyLoss</code>	$\log(1 + ((pred - target)/scale)^2)$
<code>CharbonnierLoss</code>	yumuşatılmış L1
<code>TukeyBiweightLoss</code>	eşik dışında gradyan sıfır

Sınıflandırma

`nexusloss` içinde, `LossBase`: `BCELoss`, `BCEWithLogitsLoss`, `HingeLoss`, `SquaredHingeLoss`, `ExponentialLoss`, `PerceptronLoss`, `ZeroOneLoss`, `FocalLoss`.

`nexusloss::classification` içinde, tek örnek: `CrossEntropyLoss` (label smoothing öntanımlı 0), `LabelSmoothingCrossEntropyLoss`, `KLDivLoss`.

Serbest fonksiyonlar: ikili ve kategorik çapraz entropi, sparse ve label-smoothing çeşitleri, ikili ve kategorik focal, genelleştirilmiş ve simetrik çapraz entropi, ağırlıklı BCE / CE, `effective_number_class_weights`. Hinge ailesinin hedefi -1 ve +1, BCE hedefi [0, 1] aralığıdır.

Segmentasyon

Sınıflar: `DiceLoss`, `IoULoss`, `TverskyLoss`, `FocalTverskyLoss`, `LovaszSoftmaxLoss`. Serbest: combo (Dice + BCE), boundary, Hausdorff, Lovász-hinge, Lovász-softmax. Yumuşak maskeler olasılıktır. Lovász-hinge logit ister. Hausdorff uzaklık dönüşümü çalıştırmaz; sınır mesafe haritasını çağıran verir.

Tespit

Köşe sırası `[x_min, y_min, x_max, y_max]`. Kayıplar: IoU, GloU, DloU, CloU ve gradyanları. `DetectionFocalLoss` logit alır; alfa 0.25, gama 2. `SmoothL1BoxLoss` kodlanmış offset içindir (dx, dy, dw, dh); beta öntanımlı 1. Köşe kutusu ile aynı tensör değildir.

Metrik öğrenme

`ContrastiveLoss`, `TripletLoss`, `ArcFaceLoss` (marj 0.5, ölçek 64), `CosFaceLoss` (marj 0.35, ölçek 64). Serbest: kosinüs gömme, center, n-pair, SphereFace. Açısal kayıplar normalize edilmiş kosinüs logiti bekler. Gradyanlar giriş sırasıyla art arda durur.

Sıralama

`MarginRankingLoss`, `RankNetLoss`, `ListNetLoss`, `ListMLELoss`. Serbest: BPR, yaklaşık NDCG, `soft_rank_loss` (doğrudan `approx_ndcg_loss`), `LambdaRank`. Yüksek skor daha iyidir.

Üretken modeller

Sınıflar: `AdversarialLoss`, `WassersteinLoss`, `LSGANLoss`, `VAEELBOLoss`. Serbest: hinge GAN, öznelik eşleme, döngü ve kimlik, algısal, stil, toplam değişim, difüzyon epsilon, L1 ve L2. Algısal ve stil kayıpları VGG çalıştırmaz. Toplam değişim görüntüsü satır-major, kanal-sondadır: $(y \cdot w + x) \cdot C + c$. Difüzyon kaybı gürültü ile tahminin kare farkının ortalamasıdır. Ayırt edici ve üretici gradyanları ayırır.

Öz-denetimli öğrenme

`InfoNCELoss` sıcaklığı öntanımlı 0.1'dir. `NTXentLoss` onun alt sınıfıdır. Ayrıca `BYOLoss` ve `BarlowTwinsLoss`. Serbest: VICReg ve DINO. Artırma hattı bu depoda yoktur; fonksiyon verilen benzerlik veya öznelik üzerinde kapanır.

Damıtım, dizi, nokta bulutu

`KnowledgeDistillationLoss` sıcaklığın karesi ile ölçeklenmiş KL kullanır. `FeatureDistillationLoss` ve `AttentionTransferLoss` öğretmeni eğitmez.

`CTCLoss` zaman-major logit bekler: `[zaman, sınıf]`, boşluk öntanımlı 0. İleri-geri log uzayında toplanır. `MaskedSequenceCrossEntropyLoss` maskeli token içindir. `next_sentence_prediction_loss` ikili karardır.

`ChamferDistanceLoss` en yakın nokta uzaklıklarını toplar; düz bellek, öntanımlı boyut 3.

`EarthMoverDistanceLoss` eşit kardinalite, Macar eşlemesi, $O(n^3)$. Büyük bulutta Chamfer seçilir.

Pekiştirme ve sağkalım

Sınıf	Anlamı
<code>PolicyGradientLoss</code>	$-\text{mean}(\log \pi \cdot A)$
<code>PPOClipLoss</code>	kırılmış vekil, epsilon 0.2
<code>ValueLoss</code>	değer fonksiyonu hatası
<code>EntropyBonusLoss</code>	$-H$; giriş olasılıktır, logit değildir
<code>CoxPHLoss</code>	Breslow risk kümesi; <code>observed</code> bool
<code>WeibullNLLLoss</code>	log-ölçek ve log-şekil ile negatif log olasılık

Ortam, avantaj tahmini ve sansür kodlaması çağırının verisindedir. Sansürlü Cox gözlemi paya girmez, risk kümesinde kalır.

11. Şekil ve kodlama

- BCE hedefleri [0, 1]. Hinge ailesi -1 ve +1.
- Çok sınıflı API: logit vektörü + `size_t` sınıf indeksi, tek örnek.
- Poisson tahminleri log-orandır.
- Tweedie gücü (1, 2) açık aralığında, ortalama pozitif.
- Tespit kutuları minimum x, minimum y, maksimum x, maksimum y.
- `SmoothL1BoxLoss` kodlanmış ofset içindir.
- CTC düzeni zaman × sınıf, zaman dışta, boşluk indeksi öntanımlı 0.
- Nokta bulutu düz bellek, öntanımlı nokta boyutu 3.
- Açısal marj kayıpları normalize kosinüs logiti bekler.
- Algısal, stil ve Hausdorff kayıpları hazır öznitelik veya mesafe haritası ister.
- Toplam değişim görüntüsü satır-major, kanal-sondadır.
- `EntropyBonusLoss` olasılık dağılımı alır.
- Şablon tipi tahminin tipiyle aynıdır. `float` ile `double` karıştırılmaz.

Sık hatalar

Belirti	Düzeltilme
<code>backward()</code> istisna	önce <code>forward</code> , ya da <code>gradient(pred, target)</code>
gradyan N kat büyük	indirgeme <code>Sum</code> ; <code>Mean</code> e geçin veya öğrenme hızını bölün
batch çapraz entropisi tek sayı değil	API tek örnektir; döngü çağırandır
BCE, ±1 etiketle anlamsız	±1 hinge ailesine gider
softmax sonrası çapraz entropi	fonksiyon logit ister, log-sum-exp içeridedir
CTC boyutu tutmuyor	zaman × sınıf; boşluk 0 değilse kurucuda verin
EMD yavaş	eşit kardinalite ve $O(n^3)$; büyük bulutta Chamfer
boş vektör	<code>check_same_size</code> reddeder

12. Test ve dokümantasyon

Her aile için `tests/test_*.cpp` vardır. `test_smoke.cpp` başlıkların birlikte derlendiğini ve temel çağrılarını sonlu kaldığını yoklar. Analitik gradyan merkezi farka bağlanır:

```
sayisal ≈ (L(x + h) - L(x - h)) / (2h)
h = 1e-6 , tolerans = 1e-5
```

`expect_finite` her bileşenin sonlu olduğunu ister. Keşif `gtest_discover_tests` ile yapılır. Yeni kayıp eklerken aynı kalıp yeterlidir: bilinen bir noktada skaler, sonlu farkla gradyan, boş ve aykırı span için istisna.

Site

`docs/website` bir Next.js uygulamasıdır. Metin başlıklardan otomatik üretilmez; katalog `docs/website/lib/docs/` altındadır.

- `/` genel bakış, hızlı başlangıç, kıyas.
- `/docs` 13 aile, 88 kayıt, formül, argüman, gradyan, kod.
- `/api/locale` ülke ipucuna göre dil. Türkiye için Türkçe, aksi halde İngilizce. Seçim `nexusloss-lang` anahtarında kalır.

```
cd docs\website
npm install
npm run dev
```

NexusLoss teknik referans · sürüm 0.1.0 · Muhammed Fatih Şahin · Bursa Teknik Üniversitesi, Bilgisayar Mühendisliği, 1. sınıf.
Sözleşmenin kaynak kodu `include/nexusloss` altındadır.